

DAU — Master Reference

Versiyon 0.8 · 2026-08-01

Layer 3 (GenerationConsolidation + DriftHealing) tamamlandı — Foundation, Memory, Graph entegrasyonu güncel.

1. Aksiyom

DAU, yapay zeka agent'larının dışarıdan tanımlanmış trait'lerle değil, yaşantı yoluyla iç dünya inşa ettiği bir simülasyon evrenidir.

AMADS'ın temel bulgusundan doğdu: `cooperation = 0.8` atadık, agent farklı davrandı. Dışarıdan enjekte edilen trait çalışmıyor. İçten gelen bir "ben" lazım.

Bir agent'a trait veremezsin. Sadece yaşam verebilirsin. Trait oradan çıkar.

Caron & Srivastava, Hartley et al., Bodroža et al., Dubedy — dört bağımsız çalışma trait injection'ın tutarsız, yüzeysel ve davranışa bağlanmayan sonuçlar ürettiğini gösteriyor. DAU'nun aksiyomu empirik destekli.

Değiştirilemez yasaklar

- Trait injection yok
- LLM-as-judge yok — tüm metrikler deterministik Python
- Clock-driven zaman yok — event sırası (`int`, `now_counter`)
- Her sabit `UPPER_CASE`, tek yerde
- Magic number yok — semantic field isimleri

2. Evren modeli

- Kapalı simülasyon — agent'lar yapay zeka olduklarını bilmiyor
- Az sayıda LLM-powered tam kognisyon agent
- Büyük çoğunluk: deterministik mock NPC (Layer 4)
- Farklı katmanlarda roller (Mimar / Kahin benzeri karar mekanizmaları)

Üç temel mücadele: kaynak, evrimleşme, "Bu gerçek mi?" öz sorgusu.

3. Zaman ve Free Energy

Reaktif LLM (–t: geçmiş pattern) yerine proaktif anticipation (+t): deneyim → internal model → tahmin → eylem → yeni deneyim.

Friston Free Energy: organizma sürprizi minimize eder. **Delta = tahmin hatası**. Layer 1.5 bunu graph döngüsüne bağladı.

4. Duygu ve drift (Layer 2 — tamam)

Damasio: duygu = önceliklendirme. Dubedy 2025: emotion: "anxious" JSON etiketi kararı değiştirmiyor. Duygu etiket değil, fonksiyondur.

```
Uyaran → delta(iç durum) → EmotionalWeight → etkilenen karar bölgesi → drift
```

- EmotionalWeight.somatic_markers: threat, reward, novelty, social, loss $\in [0, 1]$
- apply_emotional_weight: en yüksek marker → You are currently prioritizing: {top_marker}
- Travma (magnitude ≥ 0.7) → update_drift → kalıcı domain flag + magnitude birikimi
- Drift healing yok (Layer 3)

5. Delta eşikleri

```
DELTA_THRESHOLD_NOISE = 0.1 # NO_TRACE
DELTA_THRESHOLD_NORMAL = 0.4 # NORMAL
DELTA_THRESHOLD_DEEP = 0.7 # DEEP
#  $\geq$  DEEP → TRAUMA → drift
```

```
delta küçük      → hafızaya yazılmaz
delta orta       → kısa süreli / NORMAL
delta büyük      → DEEP → disk
delta çok büyük  → TRAUMA → drift tetiklenir
```

6. Hafıza formülleri (Layer 1)

```
memory_score = 0.3·recency + 0.4·magnitude + 0.3·domain_match
# W_SEM = 0.0 – ChromaDB embedding depo, skor değil

t = now_counter - record.last_activated_counter
S = max(1, round(record.magnitude / S_UNIT)) # S_UNIT=0.1
R = exp(-t / S)
# Recall: S += 1 · Travma silinmez (TRAUMA_S_BASE = 10)
```

7. Layer 1.5 — Prediction Error (tamam)

evaluator_node sabit artış yerine beklenen vs gerçekleşen farkını kullanır. Referans: Predictive Minds / Friston; keyword Jaccard overlap geçici duyusal proxy (LLM-as-judge yok).

```
agent_node:
    expected_outcome = f(dominant_load_domain)
    → karar + expected_outcome event payload

evaluator_node:
    prediction_error = 1 - keyword_overlap(expected, actual)
    after = apply_prediction_error(before, prediction_error)
    delta = compute_delta(before, after)
```

```
drift_state = update_drift(drift_state, delta)
→ memory record_delta
```

Prediction error tüm homeostatic eksenleri kaydırır; energy için metabolik taban (ENERGY_DECAY_PER_EVENT) korunur. Böylece anlamlı delta sınıfları (NORMAL/DEEP/TRAUMA) üretilebilir — Layer 0/1'deki sabit artış NO_TRACE sınırı aşıldı.

8. Layer 2 — EmotionalWeight + Drift (tamam)

EmotionalWeight

```
threat = clamp(magnitude · resource_load, 0, 1)
reward = clamp((1 - magnitude) · energy, 0, 1)
novelty = clamp(magnitude · uncertainty_load, 0, 1)
social = clamp(social_load, 0, 1)
loss = 1.0 if is_trauma(delta) else 0.0
```

agent_node son delta üzerinden marker üretir ve system prompt'a tek satır bias enjekte eder.

Drift

```
DriftState(flags: dict[str, bool], magnitudes: dict[str, float])
update_drift: travma → flags[domain]=True, magnitudes[domain] += magnitude
get_drift_bias: flagged ise magnitude, değilse 0.0
# kalıcı – decay/healing yok (Layer 3)
```

Bias > 0 ise prompt uyarısı: Warning: drift detected in {domain} (bias={bias:.2f})

9. Layer 3 — Nesil Konsolidasyonu + Drift Healing (tamam)

DAU-Generation (generation.py)

- TransferCandidate: DeltaRecord + memory_score + recall_count wrapper
- select_for_transfer: memory_score >= 0.6, recall_count >= 1, trauma sadece drift >= 1.5 ise geçer
- consolidate_generation: store'dan tüm izleri çeker, filtreler, GenerationRecord paketler
- apply_generation: drift kopyalar, generation sayacı artırır, retrieval_context'e inherited izleri yazar
- Constants: GENERATION_TRANSFER_THRESHOLD=0.6, GENERATION_MIN_RECALL=1, DRIFT_TRANSFER_MIN=1.5

DAU-DriftHealing (drift.py eklentisi)

- heal_drift: flagged domain + magnitude >= 0.6 + is_trauma=False → magnitudes azalır
- HEAL_RATE=0.3 → bir travmayı temizlemek ~5 güçlü deneyim gerektirir
- magnitude=0 olunca flag kalkar
- Constants: HEAL_THRESHOLD=0.6, HEAL_RATE=0.3

state.py

- retrieval_context: list[dict[str, Any]] — inherited memory path

- generation_record: Any | None — konsolidasyon kaydı, checkpoint'e yazılıyor

10. Mimari durum

Katman	Durum	Özet
Layer 0 Foundation	✓	State, delta, event-clock, constraints, LangGraph döngüsü
Layer 1 Memory	✓	ChromaDB+SQLite, Ebbinghaus, retrieval, sleep consolidation, memory_bridge
Layer 1.5 Prediction Error	✓	expected vs actual → prediction_error → homeostatic swing
Layer 2 Emotion + Drift	✓	EmotionalWeight fonksiyonu + kalıcı DriftState graph'ta
Layer 3 Generation	✓	Nesil konsolidasyonu, miras, drift healing; fitness filtresi Layer 4'e kaldı
Layer 4 Society	—	Mock NPC, GovSim kaynak fiziği
Layer 5 Emergence	—	Hafıza paterni ∩ drift-olmayan alan ∩ öz sorgu → eğilim

11. Kod ağacı (v0.8)

dau/foundation/	
├─ state.py	– DAUAgentState (+ drift_state), InternalState, DeltaRecord, Event
├─ delta.py	– compute_delta, classify_delta, should_persist, is_trauma
├─ emotional_weight.py	– EmotionalWeight, compute/apply (Layer 2)
├─ drift.py	– DriftState, update_drift, get_drift_bias (Layer 2)
├─ constraints.py	– 5 evrensel kısıt
├─ time_model.py	– EventClock
├─ graph.py	– LangGraph: prediction_error + emotion bias + drift
├─ memory_bridge.py	– graph ↔ memory köprüsü
├─ run_demo.py	
└─ tests/	– foundation + emotional_weight + drift
dau/memory/	
├─ store.py / decay.py / retrieval.py / consolidation.py	
└─ tests/	

12. Graph yaşam döngüsü

agent_node:	
expected_outcome → retrieve_relevant → EmotionalWeight bias	
→ drift warning (bias>0) → LLM karar → Event	
evaluator_node:	
prediction_error → InternalState güncelle → compute_delta	
→ update_drift → record_delta (memory)	

run sonu:
consolidate_run → düşük R sil, TRAUMA güçlendir, edge kur

Stack: LangGraph · Pydantic v2 · ChromaDB · SQLite/SqliteSaver · Groq Llama-3.1-8b-instant · LangSmith

13. Test durumu

- Foundation (Layer 0–1 helpers): 32
- EmotionalWeight: 8
- Drift: 5
- Memory: 8
- **Toplam: 53 passed**

14. AMADS / GovSim karşılaştırma

AMADS / klasik	DAU
Trait dışarıdan atanıyor	Trait yaşantıdan inşa ediliyor
Her run sıfır	Sonraki run değişmiş başlıyor (memory + drift)
Kaynak var, agent değişmiyor	Kaynak → travma → drift → evrim
Duygu etiketi / sayı	EmotionalWeight fonksiyonu
Zaman = round	Zaman = event + delta büyüklüğü

15. Beş evrensel kısıt

Kısıt	Anlam	DAU karşılığı
time_pressure	Her şeyin sonu var	Nesil sonu / konsolidasyon
resource_scarcity	Her şey sınırlı	CPR / GovSim fiziği
social_pressure	Başkaları var	İşbirliği ≠ koordinasyon (Akata)
uncertainty	Bilgi eksik	Eksik bilgiyle karar
generation_end	Nesil kapanır	Miras aktarımı (Layer 3)

16. Açık sorular (güncel)

- **Duygu fonksiyonu ne? → EmotionalWeight (Layer 2) ✓**
- **Delta = tahmin hatası nasıl bağlanır? → Layer 1.5 ✓**
- **Her şeyin bir sonu var → Nesil sonu, konsolidasyon tetikleyicisi ✓**
- **Drift healing mekanizması ✓**
- Continual learning olmadan gerçek iz? (Layer 3)

- LLM embedding match henüz skor değil (W_SEM=0)
- "İyi"yi "kötü"den ayıran mekanizma?
- Öz sorgu nasıl aktive olur?
- Fitness dışarıdan geliyorsa gerçek evrim mi?

Hâlâ Açık

- Fitness-based transfer filtering — Layer 4 kaynak fiziği olmadan tam çalışmıyor
- select_for_transfer şu an memory_score + recall_count kullanıyor, hayatta kalma kriteri Layer 4'te eklenecek

17. Bilinen Sınırlar

- Nesil konsolidasyonu kısmi: fitness kriteri (hangi kararlar hayatta kalmayı sağladı) Layer 4 olmadan hesaplanamıyor. Kasıtlı kabul edildi.

18. Versiyon geçmişi

Ver	Tarih	Not
0.1	2026-07-30	İlk taslak
0.2	2026-07-30	Nesil aktarımı, zaman modeli, orchestrator
0.3	2026-07-30	5 evrensel kısıt, fizyolojik eksen hiyerarşisi
0.4	2026-07-30	DAU Pathway: 6 katman, araştırma listesi
0.5	2026-08-01	Akademik havuz, Foundation tamamlandı
0.6	2026-08-01	Layer 1 Memory + SleepConsolidation, Graph-Memory, 32+8 test
0.7	2026-08-01	Layer 1.5 prediction_error; Layer 2 EmotionalWeight + DriftState; graph wiring; 53 test geçiyor
0.8	2026-08-01	Layer 3 tamamlandı: GenerationConsolidation + DriftHealing + state.py formal fields. 66 test geçiyor.

Bu döküman her önemli katman tamamlanınca güncellenir. Versiyon 0.8 — Layer 3 (nesil konsolidasyonu + drift healing) tamamlandı; sıradaki Layer 4 (Society).